fastPLS: accelerated SIMPLS for high-dimensional biomedical data with compiled CPU and accelerator backends

Dupe Ojo[a,†], Alessia Vignoli[b,c,†], Stefano Cacciatore[a,d,*], Leonardo Tenori[b,c,*]

[a] Bioinformatics Unit, International Centre for Genetic Engineering and Biotechnology, Cape Town 7925, Western Cape, South Africa

[b] Department of Chemistry "Ugo Schiff", University of Florence, Sesto Fiorentino, Italy

[c] Magnetic Resonance Center (CERM), University of Florence, Sesto Fiorentino, Italy

[d] Department of Integrative Biomedical Sciences, Institute of Infectious Disease and Molecular Medicine, University of Cape Town, Cape Town 7925, South Africa

†These authors contributed equally and share first authorship.

*Co-corresponding authors: Stefano Cacciatore, stefano.cacciatore@icgeb.org; Leonardo Tenori, tenori@cerm.unifi.it.

Abstract

Background and objective: High-dimensional biomedical PLS workflows, including multivariate NMR prediction and repeated validation, can be limited by the sequential cost and storage of SIMPLS. We developed fastPLS to accelerate SIMPLS while retaining de Jong's component equations.

Methods: The implementation reuses deflation products, latent quantities, coefficients, and predictions along one maximal component path, with compact prediction and optional implicit cross-covariance products. Deterministic float64 CPU IRLBA supported estimator-matched

validation against de Jong SIMPLS and independent R software. Approximate rSVD and accelerator routes were audited separately under prespecified criteria detailed in the Supplement.

Results: Deterministic fastPLS SIMPLS met the prespecified numerical tolerances in all 117 component-level comparisons. In matched single-CPU comparisons, it was faster than pls::simpls.fit on seven of nine datasets, with identical argmax accuracy and speed-up up to 8.90-fold. On NMR, qualified CPU rSVD (oversampling 20, two power iterations, seed 123) reduced 50-component SIMPLS fitting-plus-prediction time from 350.7 to 9.8 s while its predictions differed from deterministic IRLBA by relative Frobenius error 6.29e-11. A current-package hybrid float32 SIMPLS/LDA route processed one million ImageNet/DINOv2 training embeddings; at the requested 1,000-component boundary, top-1/top-5 accuracy was 0.8094/0.9393. This noncanonical single-run analysis is exploratory.

Conclusions: fastPLS reduces the computational barrier to sequential SIMPLS in large biomedical workflows while making solver and backend qualification explicit.

Keywords: partial least squares; accelerated SIMPLS; randomized SVD; CUDA; Metal; metabolomics

1. Introduction

Partial least squares (PLS) combines supervised dimension reduction with prediction and is widely used when biomedical predictors are correlated, high dimensional, or accompanied by multivariate responses [1-3]. Applications include cancer metabolomics and prediction of multiple NMR spectra from one acquisition [4-7].

Increasing spectral resolution, single-cell sample counts, foundation-model embeddings, and repeated validation make PLS computationally demanding. KODAMA, for example, repeatedly

fits cross-validated PLS discriminant models [8,9]. UNI and Prov-GigaPath illustrate the related post-extraction matrix regime in computational pathology [31,32]. The ImageNet/DINOv2 stress test assessed reproducible scalability, not biomedical validity [28,29].

Among PLS formulations, SIMPLS constructs sequential components without explicitly deflating the predictor matrix [11]. PLS-SVD provides a one-shot cross-covariance comparator [10], while OPLS and kernel PLS extend the framework to orthogonal filtering and nonlinear relations [12-14]. Existing R implementations do not jointly provide an optimized SIMPLS component path, memory-aware prediction, and compiled CPU and accelerator execution.

We present fastPLS, whose principal methodological contribution is an accelerated execution of de Jong SIMPLS. Sequential deflation, orthogonalization, and component definitions are retained, while previously computed products, coefficients, and predictions are reused through the component path. Low-rank solvers, implicit products, float32, cross-validation, LDA, and CPU/CUDA/Metal execution are supporting options around this core estimator. OPLS and kernel PLS reuse the same optimized SIMPLS engine. The GPL-3 R package uses reusable components from the MIT-licensed kodama-cpp project.

2. Materials and methods

2.1 Software architecture

The public pls() interface selects PLS family, component count, solver, backend, and, for classification, argmax or latent-space linear discriminant analysis (LDA). pls.single.cv() selects settings within one cross-validation layer; pls.double.cv() uses nested cross-validation. The audited 0.99.10 interface exposes PLS modelling, prediction, evaluation, cross-validation, score

plotting, and standalone truncated SVD; PCA and nearest-neighbour classifiers are not package APIs. Requested estimators are never silently substituted.

Let a index a component. A denotes the maximum retained component count, C the requested component-count set, and K denotes the number of cross-validation folds. Lowercase k is reserved for retrieval cutoffs.

The architecture separates preprocessing, the PLS estimator, direction extraction by IRLBA or rSVD, and CPU/CUDA/Metal execution (Figure 1). Benchmark rows recorded requested and executed estimators and rejected mismatches.

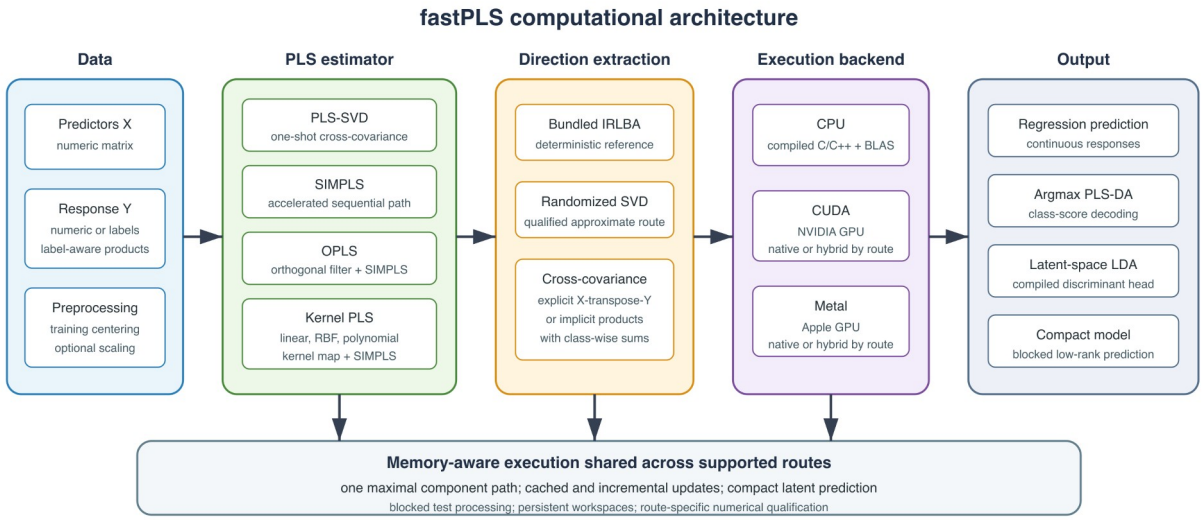

Figure 1. fastPLS architecture separating response representation, PLS estimator, low-rank solver, backend, and prediction head. Accelerator routes may be native or explicitly reported as hybrid.

CPU routines use compiled C/C++ and R-linked BLAS/LAPACK; CUDA uses NVIDIA CUDA/cuBLAS and Metal uses Apple Metal Performance Shaders. The external comparison

used one effective BLAS thread, so no multicore speed-up is claimed. Versions, build flags, residency, and thread settings are in Supplementary Tables S1 and S4a.

2.2 Accelerated SIMPLS and related PLS models

Predictors were centred and optionally scaled using training statistics; numeric responses were centred. Classification used a multivariate PLS-DA response, with class-wise products replacing dense one-hot matrices on large-class routes.

The accelerated method targets SIMPLS; PLS-SVD is retained as a one-shot comparator. Both operate on the centred cross-covariance

$$S = X^{\top} Y .$$

PLS-SVD decomposes the centred cross-covariance once at the largest valid requested rank and reuses prefixes. Its component count is bounded by cross-covariance rank and, for C centred classes, by C-1.

SIMPLS follows de Jong's sequential score, loading, orthogonalization, and rank-one deflation equations [11]. The innovation is computational: fastPLS retains deflation products, latent quantities, coefficients, and predictions incrementally, so all requested component counts are snapshots of one maximal path rather than independent refits. Each direction is still extracted from the current deflated state, and deterministic IRLBA comparisons are evaluated against prespecified numerical tolerances.

$$S_{a+1} = S_a - v_a \left(v_a^{\top} S_a \right).$$

A shape-aware route can cache X-transpose-X for score normalization and loading calculations. Compact latent factors and blocked prediction avoid retaining dense coefficient or prediction

paths. Optional implicit products and rSVD change how directions are computed or stored, not the SIMPLS deflation equations.

Algorithm 1 summarizes the component path; detailed de Jong mapping is in the Supplement.

Algorithm 1. Accelerated SIMPLS path. Direction extraction uses deterministic IRLBA or approximate rSVD; score construction, orthogonalization, and deflation follow de Jong [11].

Input: centred X; numeric, indicator, or label-aware response Y; maximum component count A; requested component-count set C; solver.
1. Define $S_0 = X^TY$ explicitly, or through the products $v \mapsto X^T(Yv)$ and $u \mapsto Y^T(Xu)$; label-aware products use class sums.
2. If $p \leq n$ and storage permits, cache $G = X^TX$. Initialize R, Q, and V as empty; set fitted values $\hat{Y}_0 = 0$.
3. For a = 1, …, A:
3.1 Extract the leading right direction g_a of the current state S_{a-1} using a fresh IRLBA solve or a fresh oversampled rSVD sketch.
3.2 Set $r_a = S_{a-1}g_a$ and $t_a = Xr_a$; divide r_a and t_a by $\|t_a\|_2$.
3.3 Compute predictor and response loadings $p_a = X^Tt_a$ and $q_a = Y^Tt_a$.
3.4 Orthogonalize $v_a = (I - VV^T)p_a$ and normalize v_a.
3.5 Deflate $S_a = S_{a-1} - v_a(v_a^TS_{a-1})$, or update the equivalent implicit operator.
3.6 Append r_a, q_a, and v_a; update $B_a = R_{1:a}Q_{1:a}^T$ and $\hat{Y}_a = \hat{Y}_{a-1} + t_aq_a^T$.
3.7 If $a \in C$, store only the requested coefficient/prediction snapshot or compact latent representation.
Output: one maximal component path with requested prefixes, rather than independently refitted models.

OPLS and kernel PLS are secondary extensions that reuse the accelerated SIMPLS core. OPLS first applies an orthogonal predictor filter [12]. Linear kernel PLS dispatches to the linear route; RBF and polynomial kernels form an n-by-n Gram matrix [13,14], restricting nonlinear models to moderate sample sizes.

2.3 Truncated SVD and memory-aware computation

Direction extraction is modular rather than the principal estimator contribution. float64 CPU fitting supports deterministic IRLBA [15] and approximate rSVD [16], which uses a Gaussian range sketch, orthonormalization, power iterations, and a reduced decomposition. Exact controls, seeds, audit thresholds, and route-specific qualification are consolidated in Supplementary Sections S13-S14 and Tables S8-S9.

For selected large shapes, fastPLS avoids materializing S and evaluates the same operator through

$$S\Omega = X^{\top}(Y\Omega), S^{\top}Q = Y^{\top}(XQ).$$

When explicit cross-covariance is large, identical centred/scaled operators are evaluated as X-transpose-(YV) and Y-transpose-(XU). Class-wise sums avoid dense indicator responses. Blocking changes storage, not the global operator.

Compact latent prediction processes large test matrices in row blocks and releases temporary scores after each block. For the ImageNet top-5 analysis, predictions were similarly blocked and classification counts were accumulated online, avoiding a full 281,167-by-1,000 double score matrix.

2.4 CPU, CUDA, Metal, and precision

Standard R matrices use eight-byte float64 values; float-package inputs use four-byte float32 values. float64 is the reference. Precision support and backend residency are route specific; the authoritative capability classifications are in Supplementary Tables S1 and S9. On Windows, a portable float-package CPU fallback supports rSVD PLS-SVD, SIMPLS, and linear-kernel SIMPLS with argmax, whereas native compiled float32 OPLS, nonlinear kernel PLS, and LDA remain unavailable.

2.5 Prediction, classifiers, and validation

Regression returns continuous predictions. Classification uses argmax PLS-DA or LDA fitted to PLS scores. LDA uses pooled within-class covariance, Cholesky solves, class priors, and deterministic trace-scaled regularization only when factorization fails [17,18]. For imbalanced classification, cross-validation can select by balanced accuracy, defined as the unweighted mean

of class-specific recalls; nested permutation testing uses the same selected endpoint rather than substituting dummy-response Q^2.

$$\Sigma = \{T^T T - \sum_c n_c \mu_c \mu_c^T\} / max(1, n - C).$$

$$\delta_c(t) = t^T w_c - \frac{1}{2} \mu_c^T w_c + \log(n_c / n).$$

Fold construction, fitting, prediction, and metric accumulation remain compiled where supported; grouped observations can be constrained to one fold. Model-selection endpoints include accuracy, balanced accuracy, R^2, Q^2, and RMSD. Hybrid OPLS, nonlinear-kernel, and Metal paths are identified explicitly.

2.6 Benchmark and reproducibility design

Twelve tasks covered metabolomics, NMR, CITE-seq, tissue and cancer omics, single-cell transcriptomics, drug response, and CIFAR-100 [7,20-27,30]. Methods used identical stored splits and training-selected component grids. CIFAR-100 followed its documented 50,000/10,000 split [30]. Classification used accuracy and Wilson intervals; multivariate regression used RMSD, Q^2, and held-out bootstrap intervals. Runtime included fitting and prediction. Isolated-process baseline and peak host RSS and process-specific GPU memory were recorded; GPU increments include runtime context, not only workspace.

A separate exploratory ImageNet/DINOv2 analysis used a fixed 1,000,000/281,167 development split of 1,024-dimensional embeddings. The split was noncanonical and previously informed component choices. SIMPLS argmax/LDA classification and raw/PCA/PLS FAISS retrieval were therefore treated as separate exploratory experiments.

Five-fold training-only selection was performed separately by PLS family; boundary selections were described as best within the grid. Accelerator speed-up was interpreted only for numerically concordant paired predictions. CUDA timing covered data transfer, execution, synchronization, and returned model or predictions; exact concordance thresholds and timing boundaries are reported in the Supplement.

The primary evidence tested whether deterministic IRLBA SIMPLS met the prespecified numerical tolerances against de Jong SIMPLS and improved runtime. rSVD was assessed separately using computational screens covering prediction error and correlation, latent-subspace agreement, decoded labels, and endpoint metrics. These screens are engineering guardrails rather than clinical-equivalence margins. Their exact definitions, tolerances, discordant counts, and audit results are reported in Supplementary Sections S12-S14 and Tables S7-S9. Deterministic IRLBA remains the confirmatory route when individual prediction changes are consequential.

3. Results

Results are organized around the accelerated SIMPLS contribution. Primary evidence comprises deterministic estimator comparison and the matched IRLBA comparison with independent implementations. rSVD, precision, and hardware results are reported separately with their numerical-audit status; the NMR and ImageNet large-scale analyses are exploratory feasibility studies rather than confirmatory evidence.

Deterministic fastPLS SIMPLS met the prespecified numerical tolerances in all 117 component-level comparisons with de Jong SIMPLS. Approximate rSVD was evaluated separately, and only fully audited settings support numerical-agreement claims; faster settings are labelled workflow-only. OPLS and kernel PLS met the prespecified tolerances in a separate deterministic reliability

study. Same-code execution ablations and direct PLS-SVD/SIMPLS shape comparisons are reported in Supplementary Tables S7a-S7b.

Repeated outer partitions showed that predictive variation exceeded timing variation and that several component selections remained boundary or rank constrained (Supplementary Table S14).

3.1 Accelerated SIMPLS compared with independent R implementations

The float64 single-CPU comparison attempted 126 external-package runs: 110 completed, 12 were package limitations, two timed out, and two errored. In the estimator-matched argmax subset, fastPLS and pls::simpls.fit [19] had identical accuracy on nine datasets; fastPLS was faster on seven, including 4.23-fold on CIFAR-100, 8.65-fold on Retina, and 8.90-fold on Tabula Muris. Matched accuracies were 0.8739 (8,739/10,000; Wilson 95% CI 0.8672-0.8803), 0.9678 (21,684/22,406; 0.9654-0.9700), and 0.8006 (40,077/50,059; 0.7971-0.8041), respectively. The separate fastPLS SIMPLS-LDA workflow reached CIFAR-100 accuracy 0.8710 in 10.118 s; because its prediction head differs, it is a workflow comparison rather than estimator-matched evidence (Figure 2; Supplementary Table S10).

Absolute process RSS was not uniformly reduced on small tasks. On CIFAR-100 it was 1.69 GB for fastPLS versus 13.09 GB for the matched reference, and on Tabula Muris 0.68 versus 2.20 GB.

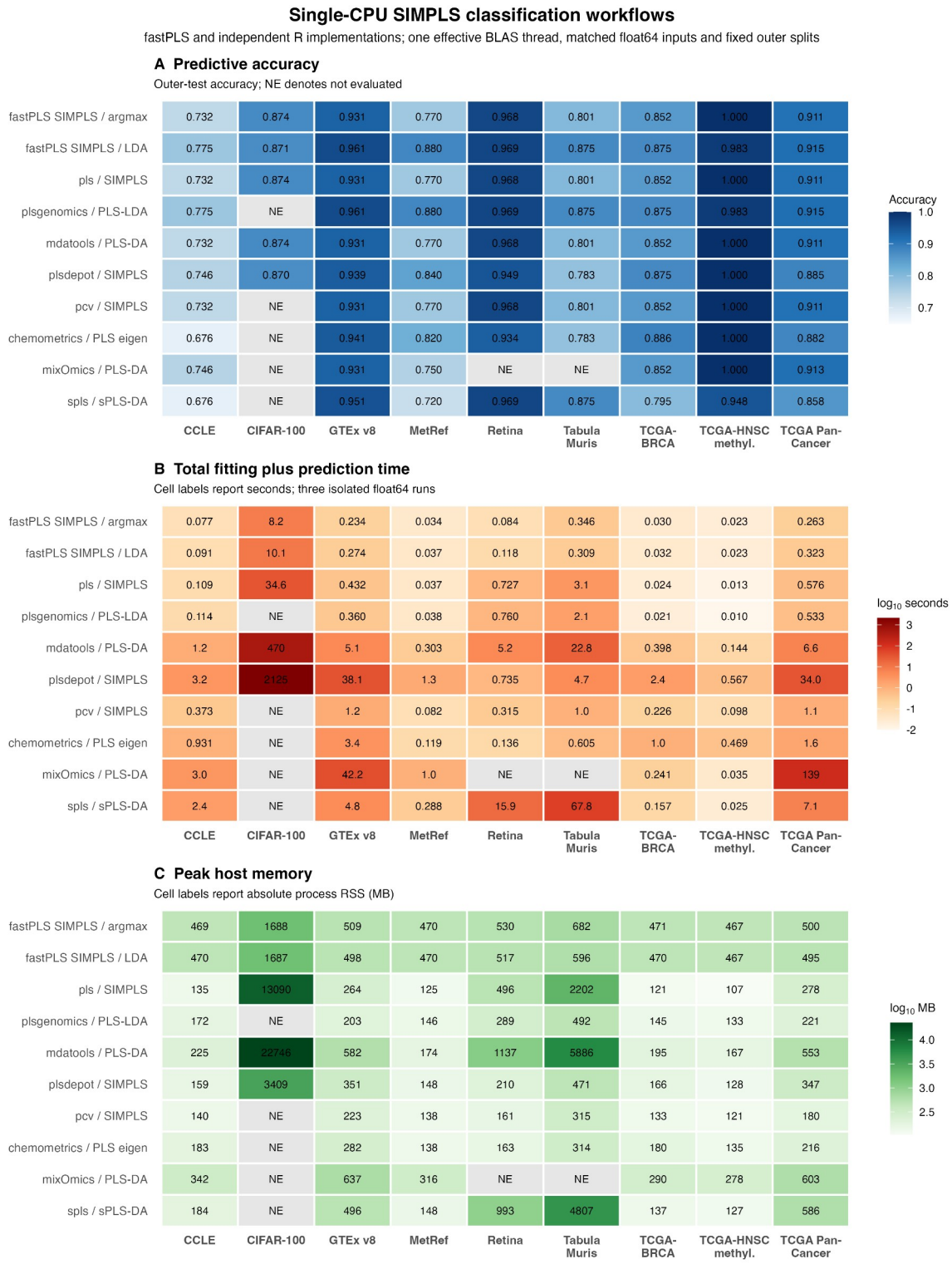
Single-CPU SIMPLS classification workflows
fastPLS and independent R implementations; one effective BLAS thread, matched float64 inputs and fixed outer splits
A Predictive accuracy
Outer-test accuracy; NE denotes not evaluated
fastPLS SIMPLS / argmax 0.732 0.874 0.931 0.770 0.968 0.801 0.852 1.000 0.911
fastPLS SIMPLS / LDA 0.775 0.871 0.961 0.880 0.969 0.875 0.875 0.983 0.915
pls / SIMPLS 0.732 0.874 0.931 0.770 0.968 0.801 0.852 1.000 0.911
plsgenomics / PLS-LDA 0.775 NE 0.961 0.880 0.969 0.875 0.875 0.983 0.915
mdatools / PLS-DA 0.732 0.874 0.931 0.770 0.968 0.801 0.852 1.000 0.911
plsdepot / SIMPLS 0.746 0.870 0.939 0.840 0.949 0.783 0.875 1.000 0.885
pcv / SIMPLS 0.732 NE 0.931 0.770 0.968 0.801 0.852 1.000 0.911
chemometrics / PLS eigen 0.676 NE 0.941 0.820 0.934 0.783 0.886 1.000 0.882
mixOmics / PLS-DA 0.746 NE 0.931 0.750 NE NE 0.852 1.000 0.913
spls / sPLS-DA 0.676 NE 0.951 0.720 0.969 0.875 0.795 0.948 0.858
CCLE CIFAR-100 GTEx v8 MetRef Retina Tabula Muris TCGA-BRCA TCGA-HNSC methyl. TCGA Pan-Cancer
Accuracy
1.0
0.9
0.8
0.7
B Total fitting plus prediction time
Cell labels report seconds; three isolated float64 runs
fastPLS SIMPLS / argmax 0.077 8.2 0.234 0.034 0.084 0.346 0.030 0.023 0.263
fastPLS SIMPLS / LDA 0.091 10.1 0.274 0.037 0.118 0.309 0.032 0.023 0.323
pls / SIMPLS 0.109 34.6 0.432 0.037 0.727 3.1 0.024 0.013 0.576
plsgenomics / PLS-LDA 0.114 NE 0.360 0.038 0.760 2.1 0.021 0.010 0.533
mdatools / PLS-DA 1.2 470 5.1 0.303 5.2 22.8 0.398 0.144 6.6
plsdepot / SIMPLS 3.2 2125 38.1 1.3 0.735 4.7 2.4 0.567 34.0
pcv / SIMPLS 0.373 NE 1.2 0.082 0.315 1.0 0.226 0.098 1.1
chemometrics / PLS eigen 0.931 NE 3.4 0.119 0.136 0.605 1.0 0.469 1.6
mixOmics / PLS-DA 3.0 NE 42.2 1.0 NE NE 0.241 0.035 139
spls / sPLS-DA 2.4 NE 4.8 0.288 15.9 67.8 0.157 0.025 7.1
CCLE CIFAR-100 GTEx v8 MetRef Retina Tabula Muris TCGA-BRCA TCGA-HNSC methyl. TCGA Pan-Cancer
log10 seconds
3
2
1
0
-1
-2
C Peak host memory
Cell labels report absolute process RSS (MB)
fastPLS SIMPLS / argmax 469 1688 509 470 530 682 471 467 500
fastPLS SIMPLS / LDA 470 1687 498 470 517 596 470 467 495
pls / SIMPLS 135 13090 264 125 496 2202 121 107 278
plsgenomics / PLS-LDA 172 NE 203 146 289 492 145 133 221
mdatools / PLS-DA 225 22746 582 174 1137 5886 195 167 553
plsdepot / SIMPLS 159 3409 351 148 210 471 166 128 347
pcv / SIMPLS 140 NE 223 138 161 315 133 121 180
chemometrics / PLS eigen 183 NE 282 138 163 314 180 135 216
mixOmics / PLS-DA 342 NE 637 316 NE NE 290 278 603
spls / sPLS-DA 184 NE 496 148 993 4807 137 127 586
CCLE CIFAR-100 GTEx v8 MetRef Retina Tabula Muris TCGA-BRCA TCGA-HNSC methyl. TCGA Pan-Cancer
log10 MB
4.0
3.5
3.0
2.5

Figure 2. float64 single-CPU SIMPLS classification workflows. Argmax rows compare matched SIMPLS estimators; LDA rows compare complete workflows with a different prediction head. Panels report accuracy, fitting-plus-prediction time, and absolute process RSS. NE denotes unavailable or incomplete runs.

3.2 Internal acceleration and low-rank solver choice

Same-code ablations on CIFAR-100, MetRef, PRISM, and Retina produced zero endpoint-metric differences and identical classifications. Compact prediction reduced incremental RSS by up to 77.7% and time by up to 1.24-fold; implicit cross-covariance products reduced RSS by up to 70.6% but were faster only in the high-response PRISM regime. This shape dependence motivates adaptive internal routing rather than universal activation (Supplementary Table S7a).

A direct matched-shape timing study further separated estimator choice from implementation cost. On one CPU, the SIMPLS/PLS-SVD time ratio ranged from 1.00 to 3.84; on CUDA it ranged from 0.92 to 0.98 across five synthetic matrix regimes. Thus, the accelerated SIMPLS path approached one-shot PLS-SVD runtime on the tested CUDA shapes, without implying that the two PLS estimators are statistically identical (Supplementary Table S7b).

Hardware acceleration remained route and shape dependent. CPU, CUDA, and Metal speed-up was summarized only for paired predictions meeting the stated concordance criteria (Figure 3; Supplementary Table S11). The exploratory one-power rSVD setting met only 101/117 audit checks and its speed results are quarantined in Supplementary Figure S1; deterministic IRLBA remains the reference. float32 approximately halved stored inputs on MetRef and PRISM but did not uniformly improve runtime, incremental memory, or agreement (Supplementary Table S9).

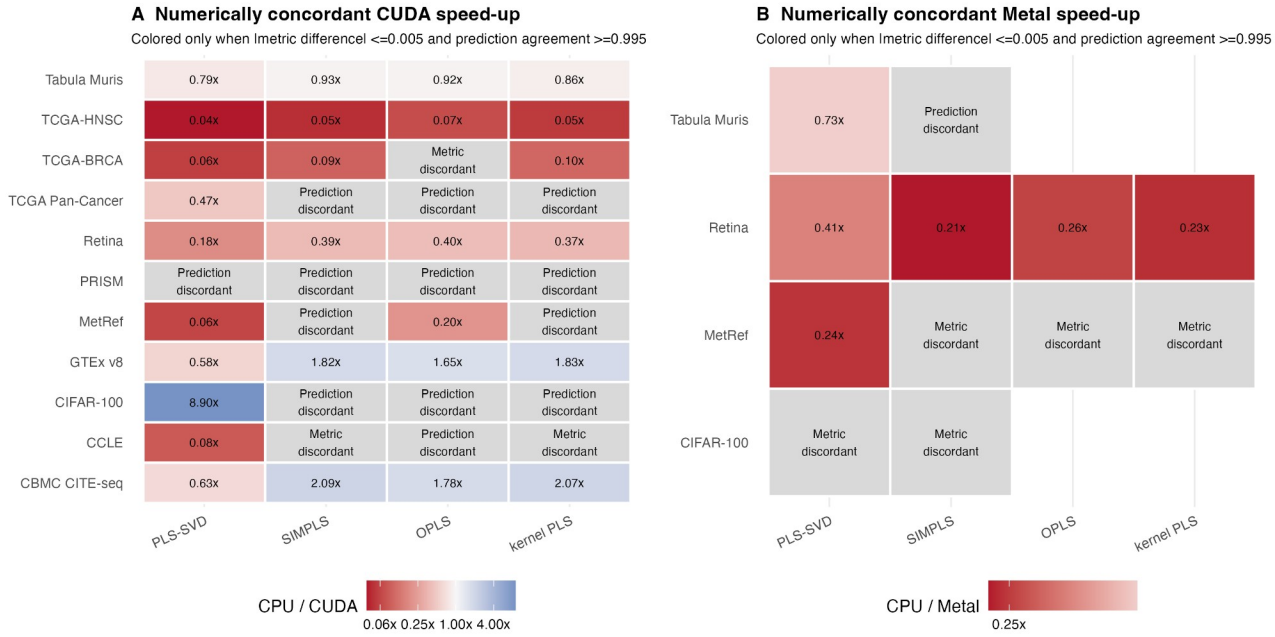

Figure 3. Numerically concordant accelerator workflows. Values are CPU/accelerator total-time ratios and are colored only when the absolute predictive-metric difference was at most 0.005 and paired-prediction agreement was at least 0.995; ratios above one favor the accelerator. Gray cells identify metric or prediction discordance, or missing paired predictions. Full paired values are in Supplementary Table S11.

3.3 Multivariate NMR prediction

NMR comprised 1,200 training and 321 held-out spectra, with 13,000 predictors and 28,355 responses. Predictor columns between 4.6 and 4.8 ppm were zeroed in training and test data as standard water-region preprocessing; responses were unmasked. Training-only one-standard-error selection retained five PLS-SVD and 50 SIMPLS components. These are family-specific predictive settings rather than a matched implementation comparison. The complete candidate grids and executable selection rule are reported in Supplementary Section S17.

At the family-selected settings, CUDA PLS-SVD and SIMPLS achieved RMSD 0.001043 (Q^2 0.98916) and 0.000759 (Q^2 0.99425), respectively. SIMPLS had lower median per-spectrum and response-wise error. Figure 4 displays held-out sample AMI-00BP-8 (index 155), whose per-spectrum RMSD under 50-component SIMPLS CUDA rSVD was closest to the median across the 321 held-out spectra. This prespecified descriptive rule did not select the best-predicted or most visually concordant spectrum.

A separate matched solver/backend analysis held family, split, float64 precision, and component count fixed. For PLS-SVD at five components, CPU IRLBA, CPU rSVD, and CUDA rSVD required 264.3, 4.74, and 0.41 s; rSVD predictions had relative error at most 3.75e-11 against IRLBA. For SIMPLS at 50 components, the times were 350.7, 9.80, and 1.51 s. CPU rSVD relative prediction error was 6.29e-11; CUDA rSVD error was 0.00566 with correlation 0.999981. All rSVD rows used oversampling 20, two power iterations, and seed 123 and met the prespecified approximate-route tolerances.

The deposited 165-component PLS-SVD/IRLBA workflow required 447.6 s and achieved RMSD 0.000710. It is shown as scientific context for the earlier Nature Communications analysis, not as an implementation-only contrast, because component count, workflow, and hardware differ.

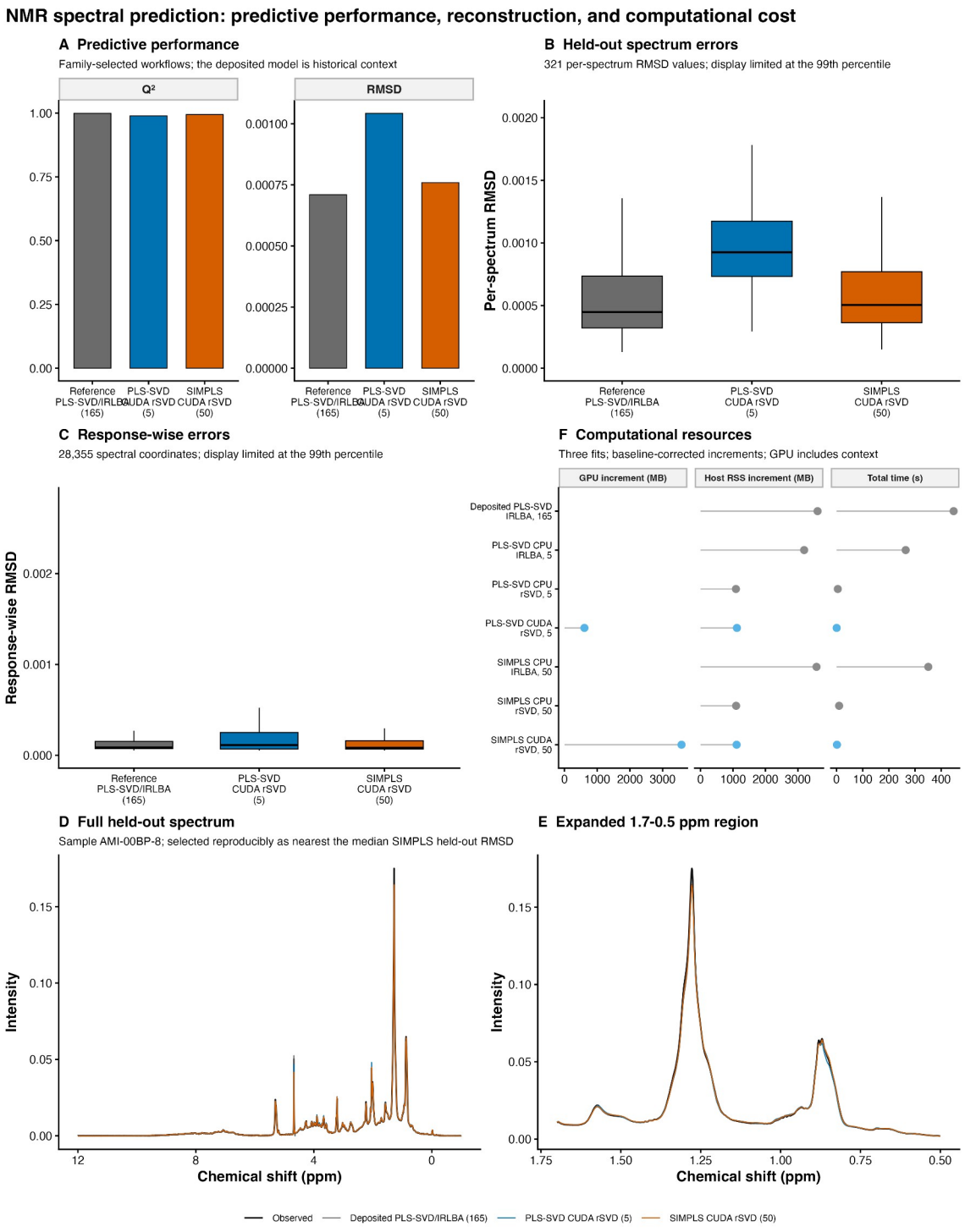

Figure 4. NMR predictive and computational analyses. Panels A-C separate family-selected held-out performance from the deposited 165-component historical context. Panels D-E overlay observed and predicted intensities for held-out sample AMI-00BP-8 (index 155), whose per-spectrum RMSD under 50-component SIMPLS CUDA rSVD was closest to the median across the 321 held-out spectra, over the full response range and 1.7-0.5 ppm expansion. Panel F reports

matched float64 solver/backend resources at fixed family-specific component counts. rSVD used oversampling 20, two power iterations, and seed 123.

3.4 ImageNet-scale supervised representation

Exploratory ImageNet experiment 1 used the current package on 1,000,000 training and 281,167 held-out 1,024-dimensional embeddings. The actual route was label-aware float32 SIMPLS with GPU rSVD and native CUDA LDA, but sequential SIMPLS deflation and score projection remained host-resident; it is therefore hybrid, not fully GPU-resident. rSVD used oversampling 20, two power iterations, and seed 123.

A single shared component-path fit evaluated requested prefixes from 100 to 1,000 components. The 1,000-component row was the prespecified boundary stress point, not a selected optimum. Top-1/top-5 accuracy changed from 0.7806/0.9278 at 100 components to 0.8094/0.9393 at 1,000 components; the latter corresponded to 227,571/281,167 correct top-1 predictions (Wilson 95% CI 0.8079-0.8108). The shared fit required 950.2 s. Peak process RSS was 17236 MB and peak device memory was 5412 MB (4122 MB above baseline). Because the split was noncanonical and the measurements were single runs, these are exploratory feasibility estimates, not confirmatory accuracy claims.

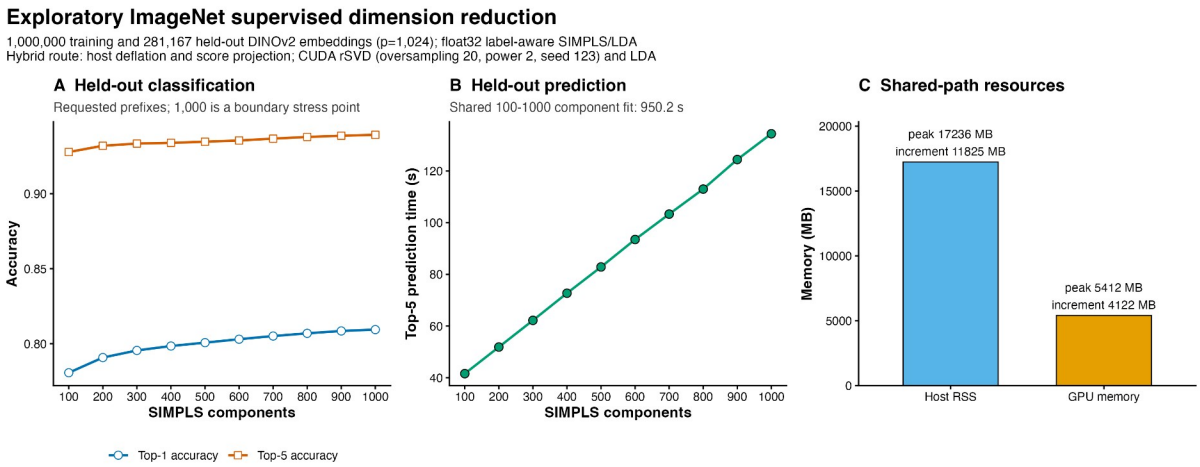

Figure 5. Exploratory ImageNet float32 SIMPLS-LDA component path. Accuracy and prefix-specific held-out prediction time are shown for requested 100-1,000-component prefixes from one shared fit; 1,000 is a boundary stress point, not a selected optimum. Resource values describe the complete shared path. The route is hybrid: SIMPLS deflation and score projection are host-resident, while rSVD range products and LDA use CUDA. Controls were oversampling 20, two power iterations, and seed 123.

4. Discussion

The principal contribution of fastPLS is an accelerated SIMPLS execution path, not a new PLS estimator. Deterministic validation showed that reuse of sequential quantities met the prespecified deterministic numerical tolerances against de Jong SIMPLS, while the external comparison showed lower runtime and, for large tasks, lower memory without changing matched accuracy.

rSVD, implicit products, float32, CUDA, and Metal are optional implementation mechanisms around accelerated SIMPLS. Qualified NMR controls met the prespecified approximate-route

tolerances, but rSVD remains stochastic and CPU IRLBA remains the deterministic reference. The million-sample ImageNet route demonstrated feasibility with current package code, while its hybrid residency, single run, noncanonical split, and lack of an estimator-matched large-scale control preclude a general accelerator or accuracy claim.

OPLS and kernel PLS demonstrate deterministic reuse of the accelerated core. NMR and ImageNet illustrate potential scale, but their rSVD results are exploratory. ImageNet does not establish biomedical validity. Limitations include finite component grids, conditional uncertainty, quadratic nonlinear kernels, approximate rSVD, and route-specific precision or accelerator disagreement.

5. Conclusions

fastPLS reduces avoidable computation and storage along the sequential SIMPLS component path while preserving the deterministic de Jong estimator within the prespecified numerical tolerances. OPLS, kernel PLS, approximate low-rank solvers, and accelerator backends extend this core contribution under route-specific validation; deterministic float64 CPU SIMPLS remains the confirmatory reference.

Ethics statement

This software study used public, simulated, or previously collected de-identified data; source-study ethics are reported in the cited publications.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data and code availability

Code and benchmark outputs are available at https://github.com/tkcaccia/fastPLS; reusable components are at https://github.com/tkcaccia/kodama-cpp. Quantitative results remain tied to fastPLS 0.99.6, base commit 6e50bd318f20289101f6b723953830aefa8b95d6, and source-archive SHA-256 c868770fee196af24bfb15b4da9fad1d7765deaf68c0ef1fd3e5a15670ecaa85. The current audited interface is fastPLS 0.99.10; its source archive has SHA-256 163ac7bd5c0c241f3817fac989e219f71b3956b388f6fcefa2f3420c45051b25. Analysis-specific scripts and archive digests are reported in Supplementary Table S15.

References

[1] Wold S, Sjostrom M, Eriksson L. PLS-regression: a basic tool of chemometrics. Chemom Intell Lab Syst. 2001;58:109-130. https://doi.org/10.1016/S0169-7439(01)00155-1.

[2] Geladi P, Kowalski BR. Partial least-squares regression: a tutorial. Anal Chim Acta. 1986;185:1-17. https://doi.org/10.1016/0003-2670(86)80028-9.

[3] Barker M, Rayens W. Partial least squares for discrimination. J Chemom. 2003;17:166-173. https://doi.org/10.1002/cem.785.

[4] Bertini I, Cacciatore S, Jensen BV, et al. Metabolomic NMR fingerprinting to identify and predict survival of patients with metastatic colorectal cancer. Cancer Res. 2012;72:356-364. https://doi.org/10.1158/0008-5472.CAN-11-1543.

[5] Cacciatore S, Wium M, Licari C, et al. Inflammatory metabolic profile of South African patients with prostate cancer. Cancer Metab. 2021;9:29. https://doi.org/10.1186/s40170-021-00265-6.

[6] Elebo N, et al. Serum metabolomic and lipoprotein profiling of pancreatic ductal adenocarcinoma patients of African ancestry. Metabolites. 2021;11:663. https://doi.org/10.3390/metabo11100663.

[7] Vignoli A, Cacciatore S, Tenori L. Deriving three one-dimensional NMR spectra from a single experiment through machine learning. Nat Commun. 2025;16:10159. https://doi.org/10.1038/s41467-025-65294-x.

[8] Cacciatore S, Tenori L, Luchinat C, Bennett PR, MacIntyre DA. KODAMA: an R package for knowledge discovery and data mining. Bioinformatics. 2017;33:621-623. https://doi.org/10.1093/bioinformatics/btw705.

[9] Cacciatore S, Luchinat C, Tenori L. Knowledge discovery by accuracy maximization. Proc Natl Acad Sci USA. 2014;111:5117-5122. https://doi.org/10.1073/pnas.1220873111.

[10] Wegelin JA. A survey of partial least squares methods, with emphasis on the two-block case. University of Washington; 2000.

[11] de Jong S. SIMPLS: an alternative approach to partial least squares regression. Chemom Intell Lab Syst. 1993;18:251-263. https://doi.org/10.1016/0169-7439(93)85002-X.

[12] Trygg J, Wold S. Orthogonal projections to latent structures (O-PLS). J Chemom. 2002;16:119-128. https://doi.org/10.1002/cem.695.

[13] Lindgren F, Geladi P, Wold S. The kernel algorithm for PLS. J Chemom. 1993;7:45-59. https://doi.org/10.1002/cem.1180070104.

[14] Rosipal R, Trejo LJ. Kernel partial least squares regression in reproducing kernel Hilbert space. J Mach Learn Res. 2001;2:97-123.

[15] Baglama J, Reichel L. Augmented implicitly restarted Lanczos bidiagonalization methods. SIAM J Sci Comput. 2005;27:19-42. https://doi.org/10.1137/04060593X.

[16] Halko N, Martinsson PG, Tropp JA. Finding structure with randomness: probabilistic algorithms for constructing approximate matrix decompositions. SIAM Rev. 2011;53:217-288. https://doi.org/10.1137/090771806.

[17] Fisher RA. The use of multiple measurements in taxonomic problems. Ann Eugen. 1936;7:179-188.

[18] McLachlan GJ. Discriminant Analysis and Statistical Pattern Recognition. Wiley; 1992.

[19] Mevik BH, Wehrens R. The pls package: principal component and partial least squares regression in R. J Stat Softw. 2007;18:1-23. https://doi.org/10.18637/jss.v018.i02.

[20] Assfalg M, et al. Evidence of different metabolic phenotypes in humans. Proc Natl Acad Sci USA. 2008;105:1420-1424. https://doi.org/10.1073/pnas.0705685105.

[21] Stoeckius M, et al. Simultaneous epitope and transcriptome measurement in single cells. Nat Methods. 2017;14:865-868. https://doi.org/10.1038/nmeth.4380.

[22] GTEx Consortium. The GTEx Consortium atlas of genetic regulatory effects across human tissues. Science. 2020;369:1318-1330. https://doi.org/10.1126/science.aaz1776.

[23] Ghandi M, et al. Next-generation characterization of the Cancer Cell Line Encyclopedia. Nature. 2019;569:503-508. https://doi.org/10.1038/s41586-019-1186-3.

[24] Hoadley KA, et al. Cell-of-origin patterns dominate the molecular classification of 10,000 tumors from 33 types of cancer. Cell. 2018;173:291-304.e6. https://doi.org/10.1016/j.cell.2018.03.022.

[25] Macosko EZ, Basu A, Satija R, et al. Highly parallel genome-wide expression profiling of individual cells using nanoliter droplets. Cell. 2015;161:1202-1214. https://doi.org/10.1016/j.cell.2015.05.002.

[26] Tabula Muris Consortium. Single-cell transcriptomics of 20 mouse organs creates a Tabula Muris. Nature. 2018;562:367-372. https://doi.org/10.1038/s41586-018-0590-4.

[27] Corsello SM, et al. Discovering the anticancer potential of non-oncology drugs by systematic viability profiling. Nat Cancer. 2020;1:235-248. https://doi.org/10.1038/s43018-019-0018-6.

[28] Deng J, et al. ImageNet: a large-scale hierarchical image database. Proc IEEE CVPR. 2009:248-255. https://doi.org/10.1109/CVPR.2009.5206848.

[29] Oquab M, Darcet T, Moutakanni T, Vo HV, Szafraniec M, Khalidov V, et al. DINOv2: learning robust visual features without supervision. Trans Mach Learn Res. 2024. https://openreview.net/forum?id=a68SUt6zFt; arXiv:2304.07193.

[30] Krizhevsky A, Hinton G. Learning multiple layers of features from tiny images. Technical report. University of Toronto; 2009. https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf.

[31] Chen RJ, Ding T, Lu MY, et al. Towards a general-purpose foundation model for computational pathology. Nat Med. 2024;30:850-862. https://doi.org/10.1038/s41591-024-02857-3.

[32] Xu H, Usuyama N, Bagga J, et al. A whole-slide foundation model for digital pathology

from real-world data. Nature. 2024;630:181-188. https://doi.org/10.1038/s41586-024-07441-w.